

Chest: A Domain-specific Language for Safe and Agile Elastic Dataflow Accelerators

Canberk Sönmez
EPFL
Lausanne, Switzerland
canberk.sonmez@epfl.ch

Mohamed Shahawy
EPFL
Lausanne, Switzerland
mohamed.shahawy@epfl.ch

Paolo Ienne
EPFL
Lausanne, Switzerland
paolo.ienne@epfl.ch

Abstract

Creating dataflow circuits with HLS tools to exploit memory- or task-level parallelism is challenging. These tools can generate functional designs easily, but optimizing them for performance often requires extensive code restructuring, pragma tuning, and even RTL integration. Writing low-level RTL, however, is rarely productive. We propose Chest, a domain-specific language that provides a thin abstraction layer over RTL for building complex dataflow circuits with elastic interfaces. Chest enables a safe and agile development flow with its novel approach and offers a rich component library to accelerate design, making such circuits more accessible on FPGAs. In this paper, we introduce Chest, outline its design principles, and present two case studies comparing it with Vitis HLS.

Keywords

FPGA, HLS, DSL

ACM Reference Format:

Canberk Sönmez, Mohamed Shahawy, and Paolo Ienne. 2018. Chest: A Domain-specific Language for Safe and Agile Elastic Dataflow Accelerators. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 11 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

High-level synthesis (HLS) tools are notable for increasing designer productivity when programming FPGAs, making reconfigurable computing more accessible. They work exceptionally well for solving a multitude of problems of deterministic nature and generate functional designs easily from software descriptions, such as C programs. However, achieving high-performance designs with HLS is challenging. The designer needs to write the code conforming to strict guidelines and use *pragmas* where necessary to guide the tool. In particular, extracting task-level or memory-level parallelism in HLS often requires fundamental restructuring of the original source code. The result no longer resembles a sequential program but a composition of simplistic *dataflow* components that work in parallel and communicate over *stream interfaces*. Of course, these dataflow components must work around the peculiarities of the

HLS tool. This not only obscures the logic of the design but also introduces significant legwork that runs against the promised productivity benefits of HLS. At the end of this engineering process, there is no guarantee that the output matches the expectations. We seek a better way to create dataflow designs.

To this end, we propose Chest, a Scala-based domain-specific language (DSL) built on top of Chisel [4] for developing agile and high-performance dataflow circuits. It introduces a thin abstraction layer on top of RTL to *ensure* that the designer gets precisely what they ask for, without going through the laborious process of writing low-level RTL. Yet, being so close to RTL allows Chest to achieve tasks that would be impossible to do in HLS. In other words, Chest has the flexibility of RTL with the productivity of HLS.

Chest introduces a novel design paradigm for constructing elastic dataflow circuits with ease. It offers a rich set of components and flexible programming interfaces, including stateful elements such as transducers and counters, control constructs like loops, and core dataflow primitives such as forks and joins—all working synergistically to simplify the development of complex circuits. AXI components further enable designers to build memory infrastructures tailored to their workloads. Chest also improves reliability by performing static checks at synthesis time and inserting runtime assertions and warnings during simulation.

We believe that Chest reached a sufficiently mature state to be released as an open-source project for the community. This paper presents Chest by providing code examples and case studies.

2 Motivation

In this section, we discuss why a lower-level abstraction like Chest is desirable in the presence of HLS tools which can also generate dataflow circuits—at least to some extent. We argue that guiding HLS tools into achieving the ideal dataflow circuit needs fundamental code restructuring, hunting for the correct pragmas and flags, and even resorting to pure RTL. This is in contrast to Chest, which aligns more naturally with the dataflow paradigm. To make our point, we consider a simple yet ubiquitous C kernel and an ideal dataflow circuit implementation of it. We implement it using Vitis HLS, which is one of the most complete and well-supported HLS tools, with all the necessary steps to get it close to the ideal. This will set the stage for our introduction of Chest in Section 3.

Listing 1 presents the `gatherSum` kernel, which sums a subset (`idx[]`) of numbers (`val[]`). This kernel can be viewed as a building block of a sparse matrix-vector accelerator. It takes a *stream* of tasks (consisting of length, indices, and values), issues memory accesses, performs floating-point additions, and outputs a stream of results. It is a memory-bound kernel, making the memory access performance critical. To this end, the circuit should use

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM

<https://doi.org/XXXXXXX.XXXXXXX>

bursts for sequential accesses (`idx[]`). A burst transfer carries many data packets per memory request and it uses the memory bandwidth more efficiently than issuing individual memory requests. For random accesses (`val[]`), it should issue as many requests as possible, since variable-latency memories, such as HBM and DDR, overlap the processing of different requests to improve the memory throughput. Memory requests of successive tasks must be issued even before the current task completes, which is especially important in case of numerous small tasks.

First, we tested a naive HLS implementation of the kernel, with minimal modification to the original source code. This implementation roughly corresponds to a finite-state machine orchestrating functional units with respect to a static schedule, as shown in Figure 1.a. We send tasks and receive results over *stream* interfaces. We also use separate memory interfaces for accessing indices and values, with ample buffer space, to help the scheduler. We observed that this design is perfectly functional, yet very inefficient. It cannot issue memory requests of different tasks concurrently, roughly having the schedule in Figure 2.a. While this design successfully issues bursts, the schedule is far from the ideal.

Next, we attempted to improve the naive design by separating it into three independent kernels, as shown in Figure 2.b. These kernels work in parallel and they are connected over stream interfaces within a dataflow region. For the `load_index` kernel, we realized that achieving the desired memory access behavior using only static HLS techniques is not possible. As shown in Figure 2.b, it cannot parallelize the bursts of different tasks. This, in turn, limits the throughput of the other kernels. While there is some overlap between different tasks, this design is not ideal either.

Finally, we disassembled the load kernels further to issue memory requests and receive responses over stream interfaces, corresponding to Figure 1.c. Now, there are address generators instead of load units. We used RTL glue logic to convert from stream interfaces to AXI, the protocol used for accessing memory. At the end, and after many hours of engineering, this mixed design achieved the ideal memory access behavior shown in Figure 2.c.

The HLS-RTL mixed design that we have described comes very close to the ideal dataflow circuit. However, it is far from the promises of HLS. We had to modify the design to such an extent that it no longer resembles the original. In a certain way, we used the HLS tool as an RTL generator, yet still employed pure RTL to achieve the desired behavior. We had to go through the Vitis HLS manual many times to find the suitable pragmas and compiler flags.

```

1 float gatherSum(uint32_t N, uint32_t idx[], float val[]) {
2   float result = 0.0f;
3   for (int i = 0; i < N; ++i)
4     result += val[idx[i]];
5   return result;
6 }

```

Listing 1: Gather-sum kernel sums a subset of an array. It features sequential and random memory accesses and floating-point reduction. An ideal implementation should use bursts for sequential accesses and issue as many random accesses as possible to improve the memory throughput.

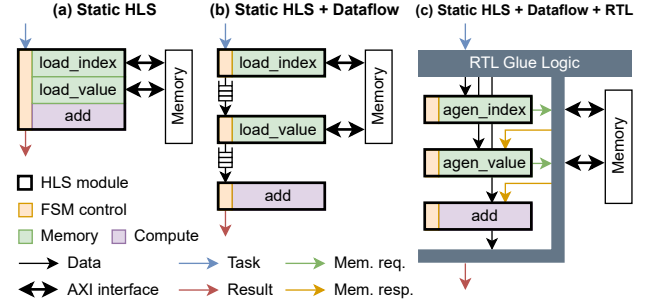


Figure 1: Step-by-step construction of the ideal dataflow design using HLS. The naive design on the left is the easiest one to implement. The second design partitions the naive one into three parallel kernels connected over streams. Neither of the first two designs achieve the desired dataflow behavior. The last design combines HLS modules with some RTL glue logic. Instead of relying on the HLS tool to access memory, it generates addresses (`agen`) and initiates memory requests explicitly. The RTL part converts the address to AXI memory requests and transfers back the memory responses. Of course, this mixed design is the most tedious one to implement, though it achieves the desired dataflow behavior.

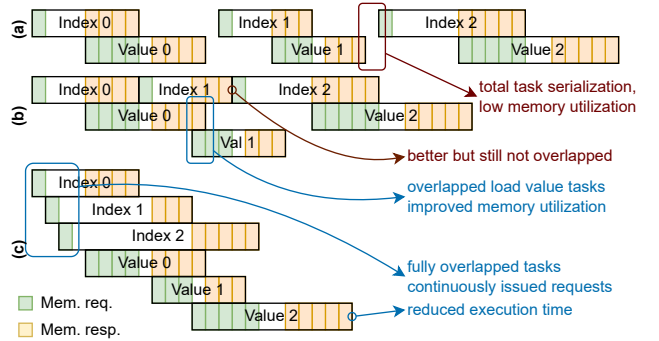


Figure 2: Representative schedules of the HLS-based designs, drawn for three tasks. We observe a gradual increase in parallelism between steps, with the best schedule achieved by the mixed HLS+RTL design. While it is obvious to the designer that the last schedule is optimal, this is not as apparent to the HLS tool. In contrast, Chext achieves this optimal schedule by default—without complicating the design process.

While achieving a basic functional design using HLS was easy, making a high-performance design was extremely challenging. This difficulty proves the need for an alternative approach—Chext, which we discuss in the following sections.

3 Chext Overview

In this section, we demonstrate the capabilities of Chext on an accelerator that implements the kernel in Listing 2. This kernel reads an array from memory, partitions it over two output arrays, and modifies each element by adding an alternating constant. We assume

that all the arrays are stored in variable-latency memory, such as HBM or DDR. As we described in Section 2, this accelerator should keep memory controllers busy by issuing as many memory requests as possible without stalling to maximize the memory throughput. It should also use bursts where feasible and have a sufficient amount of buffer space where read data resides when writes are stalled. These requirements make this accelerator a perfect candidate for the dataflow paradigm and, in the rest of this section, we will show how to implement it using Chest easily.

Chest models dataflow circuits using *components* and *elastic interfaces* connecting components [6]. An elastic interface carries two mandatory control signals, ready and valid, for flow control, and optional data signals. We say that a *token* is transferred over an elastic interface when both ready and valid are asserted at the same clock cycle. Accelerators and components have *source* and *sink* interfaces. For this example, we can assume that the accelerator has a *sourceTask* interface which carries the information related to the task (like pointers *src* and *dest1*), and a *sinkResult* interface signaling the completion. The designer's task is to instantiate components and connect them over elastic interfaces in order to implement the desired functionality.

For the given kernel, we provide an example architecture in Figure 3. This architecture contains Chest-provided read and write components, optimized for working with contiguous chunks of

```
1 int readPartitionModifyWrite(
2   int c1, int c2,           // alternating constants
3   int const *src,          // source pointer
4   unsigned len,            // length of source
5   int *dest1, int *dest2   // destination pointers
6 ){
7   assert("no aliasing among the arrays");
8   for (unsigned i = 0; i < len; ++i)
9     if (i % 2 == 0) dest1[i / 2] = src[i] + c1;
10    else dest2[i / 2] = src[i] + c2;
11   return c1 + c2; // not necessary, for demonstration purposes
12 }
```

Listing 2: An example C kernel that we implement using Chest. Use of variable-latency memory makes this kernel a perfect candidate for the dataflow paradigm.

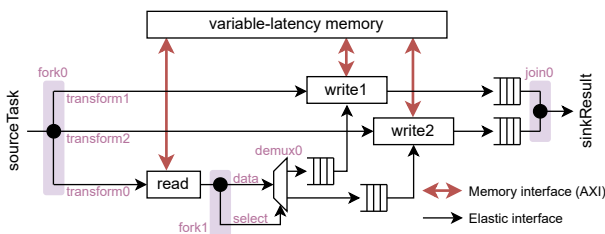


Figure 3: A dataflow architecture for the read-modify-write kernel in Listing 2. This is a memory-bound accelerator whose performance is sensitive to the memory throughput. For this reason, components accessing memory must issue as many memory requests as possible without stalling in order to increase the memory throughput. Purple annotations refer to the names of variables in Listing 3.

```
1 import chest.{elastic => e}
2
3 // component declarations are omitted: read, write1, write2
4
5 // fork allows branching a source interface to multiple sinks
6 val fork0 = new e.Fork(sourceTask) {
7   // fork(): adds a new branch to the fork
8
9   // transform connects a source to a sink interface
10  // it applies a combinational function to data
11  val transform0 = new e.Transform(fork(), read.sourceTask) {
12    // in: data of the source interface, fork()
13    // out: data of the sink interface, read.sourceTask
14
15    out.address := in.src
16    out.length := in.len
17
18    // user data is propagated to the output
19    out.user.c1 := in.c1
20    out.user.c2 := in.c2
21  }
22
23  val transform1 = new e.Transform(fork(), write1.sourceTask) {
24    out.address := in.dest1
25    out.length := (in.len >> 1) + in.len(0, 0)
26    // len(0, 0) -> Chisel syntax for the last bit of len
27    // overall, calculates ceil(in.len / 2)
28    out.user.ret := in.c1 + in.c2 // return value from kernel
29  }
30
31  val transform2 = new e.Transform(fork(), write2.sourceTask) {
32    out.address := in.dest2
33    out.length := (in.len >> 1) // calculates floor(in.len / 2)
34  }
35 }
36
37 val fork1 = new e.Fork(read.sinkResult) {
38   // in: data of the forked source interface, read.sinkResult
39
40   // select and data are combinational functions of 'in'
41   val select = in.index(0) // index is part of the read result
42   val result = Mux( // A Chisel Mux
43     select,
44     in.data + in.user.c1,
45     in.data + in.user.c2
46   ) // altered versions of the read data
47
48   // fork { `func` of `in` }: branches a new elastic interface
49   // whose data is described by a combinational function of `in`
50   // fork { T }: e.Interface[b]
51
52   // a demux routes data based to a sink on the select token
53   val demux0 = e.Demux(
54     source = fork { result }, // naming args is optional
55     sinks = Seq(
56       e.SinkBuffer(write1.sourceData, 64), // buffer sinks
57       e.SinkBuffer(write2.sourceData, 64)
58     ),
59     select = fork { select }
60   )
61 }
62
63 // join allows combining multiple source into a single sink
64 val join0 = new e.Join(sinkResult) {
65   // out: data of the joined sink interface, sinkResult
66
67   // join { `an elastic interface` }: adds a new branch to join
68   // returns the data of the new elastic interface
69   // join { e.Interface[b] }: T
70
71   // propagate the return value to the sink
72   out := join { e.SourceBuffer(write1.sinkResult, 8) }.user.ret
73   join(e.SourceBuffer(write2.sinkResult, 8))
74 }
```

Listing 3: Chest implementation of the dataflow architecture shown in Figure 3, excluding component instantiations.

memory. These components have task interfaces containing address and length fields, which are derived from the accelerator's sourceTask interface. The read result interface carries the read data and the write result interface signals completion. Please note that read and write issue high-performance bursts for accessing memory. The demultiplexer dispatches the read data to one of the write components depending on the targeted partition. Buffers at the sinks of read and write components make sure that the memory does not stall on read/write responses.

Listing 3 presents the Chext implementation of this architecture, which we invite the reader to follow alongside the discussion in this section. We would like to first emphasize that the code closely mirrors the dataflow graph shown in Figure 3—nothing more than its textual representation. Note that we have omitted the instantiations of read and write components from the code example due to space constraints. In the graph, accelerator's source interface uses `fork0` to feed `read`, `write1`, and `write2`. A *fork* creates multiple sink interfaces from a single source interface. Inside a fork, `fork()` branches the source interface into identical sink interfaces. Notice that the data of the accelerator's sourceTask interface and the sourceTask interfaces of read/write components are not compatible, they have different field names. This is fixed by adding a combinational circuit in between, handled by `transform0`, `transform1`, and `transform2`. A *transform* couples a source interface and a sink interface with an arbitrary combinational function between data signals. Inside a transform, `in` and `out` fields refer to the data of the source and sink interfaces respectively.

The result interface of `read` contains not only data, but also extra control information, such as `index`, enumerating data tokens of the task, and `user`, propagating information from the task to data tokens. The least significant bit of the index determines the targeted partition. User data allows passing the constants (see `transform0`) to calculate the final result later (see `result`). Demultiplexer `demux0` routes the result to the correct destination. It has two source interfaces, `source` and `select`, and multiple sink interfaces `sinks` (`write1` or `write2`). Both source interfaces are derived from the result interface of `read` using `fork1`. Inside a fork, `in` field allows accessing the data of the source interface, in order to create elastic interfaces with desired data. For example, `fork { result }` creates an elastic interface whose data is described by signal `result`, which is a combinational function of `in`. There are two buffers of size 64 at the sinks of `demux0`, instantiated with `SinkBuffer`.

The accelerator completes a task when it receives results from both `write1` or `write2` and sends a result token. This behavior is achieved by `join0`. A *join* synchronizes multiple source interfaces into a single sink interface. Inside a join, the `out` signal refers to the data of the sink interface. `join { interface }` adds a new source interface to the join and returns its data in order to initialize out. In the example, we also instantiate buffers at the sinks of write modules using `SourceBuffer`, each having a capacity of 8.

Our approach to dataflow design may appear unusual at first, especially to those unfamiliar with Scala; however, it is remarkably faithful to the schematic, as an RTL implementation should be.

4 Designing with Chext

In this section, we first elaborate on the design principles of Chext. Then, we discuss a select subset of Chext components due to space constraints. For sake of completeness, Table 1 provides a comprehensive list of almost all components available in Chext. Black box models of some Chext components are given in Figure 4.

Why Scala and Chisel? Scala is a powerful language that supports both object-oriented and functional programming paradigms. Despite being a strongly-typed language, Scala features strong type inference and pattern matching capabilities, boosting Chext's expressive power. Chisel is a mature hardware-description language that has gained a respectable user base over the years. It treats hardware as first-class Scala objects and, combined with the functional programming facilities of Scala, this allows the designer to create parametrized designs and complex hardware topologies.

Signaling. Chext builds on the principle of data-is-control, which means that tokens not only carry data to be processed but also the control information used by components. For components generating multiple data tokens per task, *first/last signaling* helps distinguish data corresponding to different tasks. *Index signaling* helps enumerate data tokens of a given task. *Valid/skip/zero signaling* marks data tokens that should not be considered. *User signaling* propagates information from the task interface to the result interface. *Destination/tag/select signaling* is used to identify the destination or the source of the data token. *Operand signaling* marks the part of the token used for computation. Chext components are customizable: users can directly specify which data fields (or their combinational functions) drive signaling.

Handshake rules. Chext's elastic interfaces employ the following *handshake protocol*. (1) Valid can be asserted at any time; however, once asserted, it cannot be deasserted until the token transfer is complete (i.e., ready asserted). The reason is that the downstream logic is allowed to alter its internal state once valid is asserted, even if ready has not been asserted yet. (2) Valid is not allowed to wait for ready. It is forbidden to have a combinational path from ready to valid. Ready is allowed to wait for valid; yet, it should be asserted whenever possible, even without valid being asserted, to ensure token transfers at full throughput.

Connection Primitives. The *connect operator* (`:=>`) connects a source interface to a sink interface of same type by coupling ready, valid, and data signals with no modifications. Transform also connects interfaces, possibly of different types, by applying a combinational function to the data signals. *Stall* and *drop* implement

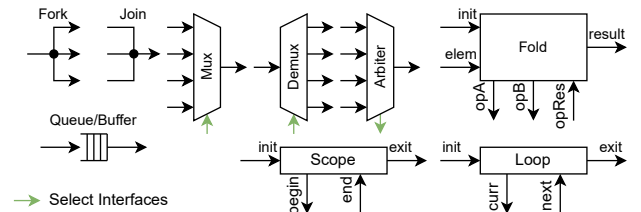


Figure 4: Black box representations of a select subset of elastic components. We omit source and sink prefixes from interface names due to space constraints.

Table 1: A list of (almost) all Chest components. They are categorized with respect to Scala namespaces they reside. DA stands for deadlock avoidance, signaling that the component is used for this purpose.

chest.elastic (elastic components)			
Connect, Transform, Stall, Drop, Fire	Transducer	Transaction (<i>BlueSpec-style rules</i>)	Switch (<i>models switch-case statement</i>)
Fork, Join Mux, Demux, Arbiter	Fold, Scope, Loop	Wrapper (<i>wraps non-elastic pipelines</i>)	Const, NullSource, NullSink
Queue, SinkBuffer, SourceBuffer	Response Buffer (DA)	Count (<i>complex counting logic</i>), Repeat	Counter (<i>counts up and wraps around</i>)
chest.amba (AXI4 full, lite and stream support)		chest.ldstr	chest.float
Mux, Demux (<i>address-based routing</i>)	Downscale, Upscale (<i>data width conversion</i>)	Load, Store (<i>non-bursting memory access</i>)	ElasticAdd
IdMux, IdDemux (<i>ID-based routing</i>)	Widen (<i>narrow-to-wide transaction conversion</i>)	chest.stream	ElasticMultiply
Protocol converter	Unburst (<i>bursts to single transactions</i>)	Read, Write (<i>bursting memory access</i>)	(based on Berkeley HardFloat library)
ResponseBuffer (<i>stall transactions to ensure enough buffer space available, DA</i>)	WriteBuffer (<i>stall AW packets until all W packets are received, DA</i>)	chest.memory (BRAM memory and controllers with elastic and AXI4 interfaces)	

conditional stalling and dropping tokens. Connection primitives support triggering logic on *fire*, i.e., during token transfer.

Fork and Join. Chest uses *eager forks* (EFork), which avoid combinational loops, and *lazy joins* (LKJoin), which are resource efficient, by default [11]. It is possible to also use *lazy forks* when the designer knows that no combinational loops are created, for instance, when a fork followed by a queue.

Queue. A *queue* temporarily stores tokens. It breaks combinational paths, prevents combinational loops and deadlocks, and ensures compliance with the handshake protocol. It optionally couples valid signals (flow or bypass mode) and/or ready signals (pipeline mode), in order to optimize for latency and/or throughput.

Mux, Demux, and Arbiter. A *mux* (multiplexer) has multiple source interfaces, one sink interface, and a select source interface carrying indices for the sources. It transfers tokens from the selected source to the sink and supports last signaling: one select token may control the transfer of multiple data tokens until the last, which depends combinationaly on the sink data and index.

A *demux* (demultiplexer) has one source interface, multiple sink interfaces, and a select source interface. Its behavior is the opposite of a mux: tokens from the source are sent to the selected sink. Demux also supports last signaling.

An *arbiter* has multiple source interfaces, a single sink interface, and a select sink interface. It transfers one token from a chosen source to the sink. Unlike a mux, it is nondeterministic because the timing of source tokens affects the choice. The arbiter uses locking choice logic, compliant with the handshake protocol: once a source is chosen, it remains selected until the transfer occurs. The chosen index is sent out on the select sink interface.

Stateful components. It is often necessary to couple elastic interfaces with finite-state machines to implement complex control logic. This may seem straightforward, but it requires correct output and transition logic respecting the handshake protocol. Chest provides components to support finite-state machines safely.

A *transducer* is a generic stateful component that implements a finite-state machine with 4 actions: (1) *accept* dequeues the token from the source and enqueues it to the sink, (2) *stall* keeps the token at the source, (3) *consume* drops the the source token, (4) *produce* enqueues a token to the sink without dequeuing the source token. Each of these actions are associated with a state transition; for example, `accept{ state := state + 1.U }` ensures that the state

transition happens only when `source.valid && sink.ready`. It also asserts `sink.valid` without depending on `sink.ready` to respect the handshake protocol. At a given clock cycle, the transducer is allowed to take exactly one action and this condition is checked in simulation. Listing 4 provides an example for transducer.

Fold. A *fold* performs the reduction of multiple source tokens into a single sink token by applying a binary operator to them. It has a source interface for receiving elements to reduce, a source interface for the initial element, and a sink interface for the result. Unlike transducer, which can also be used to implement a similar behavior, fold invokes the operator over elastic interfaces. This allows fold to use variable-latency or multi cycle operators. It keeps reducing until the *last* element and skips *zero* elements.

Scope. A *scope* defines a virtual boundary, inside which it allows only a single token at a time. It triggers logic, such as initializing registers, when a token enters and leaves. Scope is useful when (1) throughput of a token is not important, like a long running task consisting of many smaller tasks, and (2) components share a state that is constant throughout the lifetime of the scope. In the latter case, the designer does not need to propagate the shared state from one component to the next elastically (for example, using user signals), potentially reducing resource utilization.

Loop. A *loop* emulates for and while loops. Loop exposes a *current state* sink interface and a *next state* source interface. The

```

1 val transducer = new e.Transducer(source, sink) {
2   val state = RegInit(0.U(32.W))
3   val incr = Mux(in.marked, 1.U, 0.U) // +1 if marked
4   out := state
5
6   packet { // defines the logic triggered when source is valid
7     when (state > 0.U) {
8       when (in.last) { accept { state := 0.U } }
9       .otherwise { consume { state := state + incr } }
10    }.otherwise {
11      when (in.last) { accept {} }
12      .otherwise { consume { state := state + incr } }
13    }
14  }
15 }

```

Listing 4: Transducer integrates finite-state machines with elastic interfaces. In this example, it is used to count marked tokens and reset on the last token.

designer passes the current state token through elastic components to calculate the next state token. If the next state signals the end-of-the-loop, it leaves the component; otherwise, it is cycled inside. Similar to scope, loop allows only a single state token inside.

```

1 // (a) Fold example, sum floating point numbers
2 val fold = new elastic.Fold(
3   source = sourceElem,
4   sourceInit = elastic.Const(genFp32.zero),
5   sink = sinkResult
6 ) {
7   operand { _.number } // operand, used by the operator
8   last { _.last }      // last, signals when to stop folding
9
10  // operator
11  val fpAdd = Module(new float.ElasticAdd(genFp))
12
13  sinkOpA := fpAdd.sourceInA // connections using :=
14  sinkOpB := fpAdd.sourceInB
15  fpAdd.sinkOut := sourceOpRes
16 }

1 // (b) Scope example, read-modify-write a number
2 val regConstant = Reg(UInt(32.W))
3 val regAddr = Reg(UInt(64.W))
4
5 val scope = new e.Scope(sourceTask, sinkResult) {
6   init {
7     regConstant := in.constant
8     regAddr := in.wrAddr
9   }
10  exit { /* do nothing */ }
11 }
12
13 val x0 = new e.Transform(scope.sinkBegin, load.sourceTask) {
14   out.address := in.rdAddr
15   out.user := 0.U /* not used */
16 }
17
18 val fork0 = new e.Fork(load.sinkResult) {
19   // it is safe to access regConstant and regAddr
20   // they are guaranteed not to change throughout the scope
21
22   fork { _.data + regConstant } :=> store.sourceData
23   val x0 = new e.Transform(fork { regAddr }, store.sourceTask) {
24     out.address := in
25     out.user := 0.U /* no need to use the user signal */
26   }
27 }
28
29 val x1 = new e.Transform(store.sinkResult, scope.sourceEnd) {
30   out := 0.U
31 }

1 // (c) Loop example, multi-cycle division
2 val loop = new e.Loop(sourceTask, sinkResult) {
3   end { (state) => state.done } // exit condition
4 }
5
6 val transform0 = new e.Transform(loop.sinkCurrent,
7   loop.sourceNext) {
8   when (in.reminder >= in.divisor) {
9     out.reminder := in.reminder - in.divisor
10    out.quotient := in.quotient + 1.U
11    out.done := false.B
12  }.otherwise {
13    out := in
14    out.done := true.B // triggers quitting the loop
15  }
16 }

```

Listing 5: Fold, scope, and loop examples.

Internals of fold, scope, and loop are shown in Figure 5 and Listing 5 provides code examples for them.

Resource Sharing. *Resource sharing* reduces FPGA resource utilization and synthesis time by reusing expensive, underutilized components such as floating-point adders. If sharers access the shared component in order, *deterministic resource sharing* can be implemented as shown in Figure 6.a. Otherwise, it is possible to implement *nondeterministic resource sharing* as depicted in Figure 6.b. The queue capacity in this case must match the maximum number of outstanding tasks the shared component can process. *Response buffer*, shown in Figure 6.c, implements credit-based flow control to avoid possible deadlocks with nondeterministic resource sharing.

AXI Components. Improving the memory access performance is one of the major goals of Chext. To this end, Chext provides a vast array of AXI components, as listed in Table 1. The designer can use these components to construct the memory infrastructure that best suits the given workload. We emphasize that, since AXI is an elastic protocol, these components can be implemented elegantly with Chext and verified easily [2]. Chext also has components that abstracts away the protocol details of AXI, such as read/write for bursting and load/store for nonbursting accesses.

Interconnect Design. Chext provides building blocks of an interconnect rather than fixed crossbar and bus modules, allowing the user to construct custom interconnect topologies flexibly. A *demultiplexer* allows a single master interface to drive multiple slave interfaces, the targeted slave interface is calculated by an *address decode* function. A *multiplexer* performs the opposite task, it enables sharing of a single slave interface among multiple master interfaces. Figure 7 shows a few of possible configurations of these components. The designer can combine multiple interfaces into

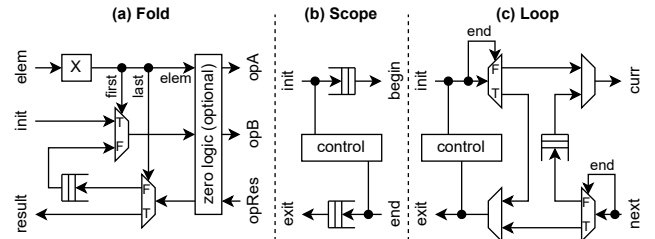


Figure 5: Internals of fold, scope, and loop. They use simpler elastic components in their construction. Scope and loop both use *stall* and *fire* to implement the control logic, ensuring that there is only a single token inside. Fold uses a transducer (X in the diagram) on elements to mark the first tokens.

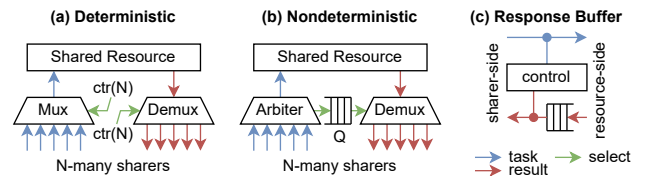


Figure 6: Resource sharing schemes and response buffer to avoid deadlocks in case of nondeterministic resource sharing.

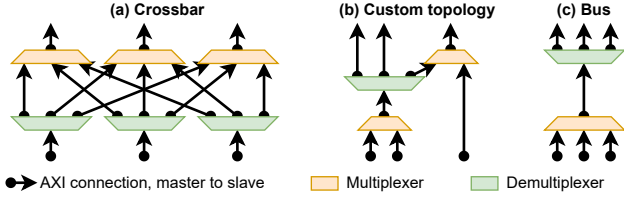


Figure 7: AXI components provided by Chext allow creating custom interconnect topologies best suiting the workload.

one with a multiplexer to save resources or isolate an interface to provide it with more bandwidth and lower the latency.

5 Case Studies and Evaluation

In this section, we implement two example kernels using Chext and statically-scheduled Vitis HLS (version 2025.1). These kernels are representative of real-world situations; they have memory-level parallelism, task-level parallelism, or both. We will compare their implementations in terms of performance and resource utilization.

To conduct our experiments, we used Chext’s simulation framework based on SystemC [3] and Verilator [17]. For a fair comparison, both Chext and Vitis HLS designs are tested using this framework, despite the fact that Vitis HLS has its own simulation tools. Furthermore, we employ Chext’s floating-point primitives in both cases. These primitives are based on Berkeley’s HardFloat library [8]. We use Vivado out-of-context synthesis for resource utilizations at 333 MHz on Alveo U55C.

5.1 Sparse Matrix-Vector Multiplication

We construct a sparse matrix-vector accelerator (SpMV) that encodes the sparse matrix using a *compressed sparse rows* (CSR) format consisting of three vectors: (1) column indices, (2) values, and (3) row lengths. Both the dense vector and the sparse matrix have 32-bit integer or IEEE 754 floating-point data [1]. CSR vectors use bursting accesses and the dense vector is accessed randomly.

Design. Figure 8 shows the architecture of the Chext-based SpMV accelerator, which consists of four stages. First, the *read memory* stage streams in CSR vectors using bursts and loads the elements of the dense input vector based on column indices. All these accesses go through an AXI multiplexer, response buffers ensure that the shared memory interface does not cause deadlocks. Next, the *prepare data* stage multiplies the elements of the input vector with values of the sparse matrix, forming a product stream. Then, using the elements of the row lengths vector elements of the product stream are marked with a last bit to signal end-of-the-rows. We use the term “row of products” for referring to this last-marked stream of numbers. This is done by joining the output of count, which has a last signal, with products. This stage is also responsible for handling zero-length rows correctly. If a row length is zero, an explicit zero is inserted. The demux/mux pair chooses the correct behavior based on the row length. The *compute stage* sums individual products in a row to calculate the output vector element, which we will talk about later in more detail. Finally, the *write memory* stage streams out the output vector using bursts.

Compute stage. We can choose from a variety of architectures for the compute stage. The simplest option is using a single fold and a floating-point adder to sum rows of products. However, the throughput of this scheme is limited by the latency of floating-point addition, which is 6 in our case. Therefore, both variants shown in Figure 9 employ multiple folds to compensate for the limited throughput. These folds share a single floating-point adder because a single fold cannot fully utilize a single adder. There are 8 fold components each, though we only show 4 of them.

In the first variant, the demux uses the last signal to assign an entire row of products to a single fold, in a round-robin fashion. Individual rows likely have different lengths; therefore, we employ nondeterministic resource sharing. The mux at the output of the circuit collects the results in order. Note that buffers before and after folds must be large enough to store an entire row, at least on average. Otherwise, the demux might stall on a row and fail to deliver data to its other sinks, reducing the overall throughput.

The second variant has a different strategy that distributes the products of a single row among eight folds, in order. It has a reduction tree at the end to sum the results coming from individual folds. It uses 8 floating-point adders in total (7 from the tree), in

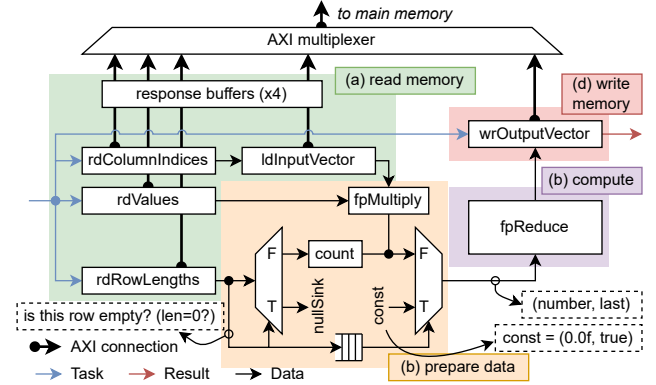


Figure 8: Architecture of the Chext-based SpMV accelerator. It merely connects simple Chext components to implement the desired functionality. The Scala code that generates this circuit is a textual representation of this dataflow graph.

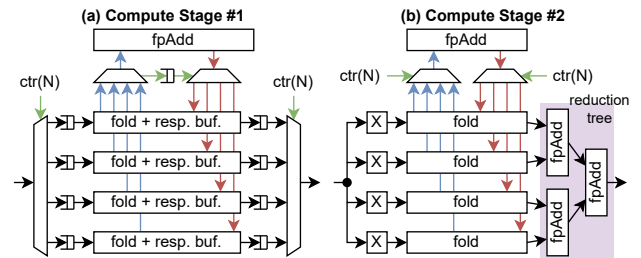


Figure 9: Two possible compute stage architectures to sum floating point numbers. Variant 1 sends distinct rows-of-products entirely to separate folds. On the contrary, Variant 2 distributes the individual products in a row among folds.

contrast to only one in the first architecture. Transducers, shown as X, discard 7 out of 8 tokens for distributing numbers among folds. However, they always propagate the last token to signal completion. This variant employs deterministic resource sharing.

We also test a third compute stage variant generated by Vitis HLS, a statically-scheduled design shown in Listing 6. Due to the simple and deterministic nature of the problem, we expect this kernel to perform well. The rest of the design is kept in Chext.

Memory latency experiment. We compare statically-scheduled Vitis HLS and Chext implementations as the memory latency varies. To this end, we designed an experiment where a simulated memory supporting up to 32 outstanding read and 32 outstanding write requests is connected to the accelerator. Performance is measured in terms of cycles per nonzero element of the sparse matrix. Ideally, increasing the memory latency should not affect the execution time as long as the it is hidden by memory-level parallelism. A deterioration is expected only after a certain latency that exceeds the available parallelism. Note that the accelerators we compare only have a single AXI interface for accessing memory. We also use integer arithmetic instead of floating-point arithmetic to isolate the effect of memory accesses from computation. Sparse matrix size is 1024×1024 , with 32 nonzero elements per row.

We present the results of the memory latency experiment in Figure 10. The Chext-based accelerator is the best performing one, with only 3 cycles/nonzero for memory latencies less than 32 cycles. This is in fact the best possible result because a single memory interface is effectively shared among three memory read operations: column indices, values, and the input vector. The effect of accessing row lengths is negligible as it is accessed much more rarely.

```

1 void computeStage(stream<Input> &source, stream<float> &sink) {
2     float partial_results[8];
3     bool stopped = false;
4
5     for (int i = 0; i < 8; ++i)
6     #pragma HLS UNROLL
7         partial_results[i] = 0.0;
8
9     while (true) {
10    #pragma HLS PIPELINE rewind
11        for (int i = 0; i < 8; ++i)
12            if (!stopped) {
13                Input in = source.read();
14                partial_results[i] += in.number;
15                if (in.last) stopped = true;
16            }
17
18        if (stopped) break;
19    }
20
21    float output = 0.0;
22
23    for (int i = 0; i < 8; ++i)
24    #pragma HLS UNROLL
25        output.result += partial_results[i];
26
27    sink.write(output);
28 }

```

Listing 6: Compute stage architecture designed using static HLS. Vitis HLS should create a circuit that works well, given the simplicity of this kernel.

There are two variants of statically-scheduled Vitis HLS accelerators. One variant sets the latency parameter of the memory interface to 100 cycles, while the other one does not. In theory, setting this parameter should help the scheduler account for the maximum memory access latency. We observe that it provides latency-insensitivity until around 100 cycles of latency. Compared to the no-latency variant, performance is worse for shorter memory latencies but better for longer latencies. The likely reason is that the scheduler always assumes the worst latency. The no-latency variant, in contrast, is always sensitive to latency, as it cannot effectively exploit memory-level parallelism.

We conclude that using the latency parameter optimizes the static HLS design only for a limited set of conditions—often at the cost increased resource utilization—while never achieving the best performance. In contrast, the Chext design works optimally across a broader range of conditions, without the need to tune hyperparameters as in static HLS.

Sparsity experiment. We measure the performance of statically-scheduled Vitis HLS and Chext designs while varying the sparsity, defined as the number of nonzero elements per row. As mentioned earlier, the compute stage is bottlenecked by floating-point additions and its throughput is therefore sensitive to sparsity. To avoid memory-related effects, the memory latency is set to 1 cycle. Moreover, vectors are accesses over separate AXI interfaces. The sparse matrix has a size of 1024×1024 .

Our results are shown in Figure 11. We first observe that the Chext design with a single fold (v.0) behaves predictably: it approaches to the floating-point adder latency (6 cycles) as the matrix becomes denser. In contrast, the statically-scheduled Vitis HLS design performs much worse than the equivalent Chext design for sparse matrices. The static scheduler likely assumes the worst case scenario, which hurts the throughput for sparse matrices.

Among the compute stage variants with 8 folds, the first one (v.1) performs the best for sparse matrices. As described earlier,

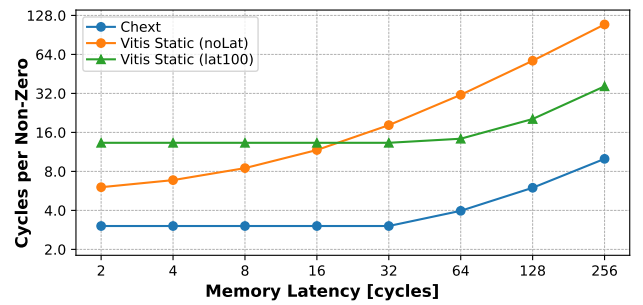


Figure 10: SpMV accelerator performance as the memory latency is varied. Chext always achieves superior performance than both statically-scheduled designs. In fact, its performance is optimal at 3 cycles/nonzero because the same memory interface is shared among 3 accesses. The available memory bandwidth is insufficient to cover the latency beyond 32 cycles. The latency parameter on Vitis HLS designs only shifts the optimal working conditions of the generated circuit. In contrast, the Chext design is always optimal.

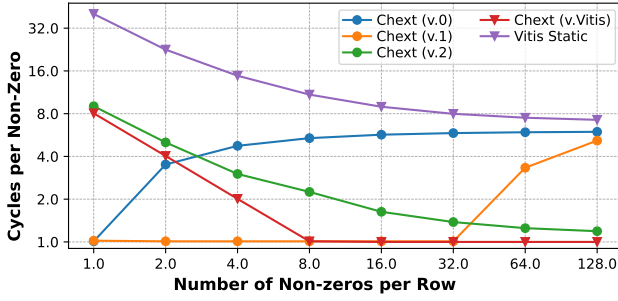


Figure 11: Performance evaluation from the compute perspective. We vary the sparsity of the matrix. Statically-scheduled Vitis HLS design is outperformed by all the Chext designs. Variant 1 outperforms other design for more sparse matrices. Variant 3 that combines Chext and Vitis HLS performs the best for dense matrices.

```

1 void histogram(
2     uint32_t len, uint32_t *features,
3     uint32_t *weights, uint32_t *histogram) {
4     for (uint32_t i = 0; i < len; ++i)
5         histogram[features[i]] += weights[i];
6 }
    
```

Listing 7: The histogram kernel. Features and weights are accessed sequentially and the histogram is accessed and modified randomly.

this variant performs worse for dense matrices because its limited-capacity buffers start to stall. Compared to the first variant, both the second (v.2) and third variants (v.Vitis) start worse for sparse matrices. This is due to the latency of the reduce tree and the pessimistic static schedule. The third variant that combines Chext and Vitis HLS performs the best for denser matrices.

Synthesis results. Synthesis results for the SpMV accelerator are available in Table 2. The resource utilization of the Vitis design and Chext design are often comparable. While the second compute stage variant has significantly more resource utilization, it could be optimized further with resource sharing in the reduce tree.

5.2 Histogram

Our next case study involves the histogram kernel shown in Listing 7. Weights corresponding to same features are accumulated in the histogram. features and weights are accessed sequentially, where as histogram is accessed randomly and also modified.

Design. Chext implementation of the histogram kernel is interesting because it needs to handle read-after-write hazards. For this reason, connecting a load, an adder, and a store serially does not work. Note that these load and store modules work in parallel, which is in fact the reason why Chext is superior to static HLS. If the same feature occurs twice consecutively, the load module might receive the stale value of the histogram from the memory, before store manages to complete updating it. Fortunately, this problem is precisely a use case of Chext’s scope. By guarding these three components with it, read-after-write hazards are prevented.

Using scope solves the read-after-write hazards at the cost of reduced memory throughput. To address this, we propose the dataflow architecture in Figure 12 that parallelizes memory accesses effectively. It consists of three stages. The first stage has two read components to access features and weights using high-performance bursts. These feature and weight streams are then concatenated, marked with a last signal, and sent to the second stage. In the second stage, there is a configurable number of load-add-store units that use scope as described earlier to update the histogram. The demux distributes feature-weight pairs among the load-add-store units depending on the feature. This mechanism ensures that the same feature always processed by the same module, avoiding data races across modules. The mux collects the load-add-store results in order. These result tokens carry the propagated last signal. In the third stage, a drop component drops results that are not marked last. The surviving token signals the completion of the overall calculation.

Memory latency experiment. We sweep the memory latency and measure the performance of statically-scheduled Vitis HLS and Chext implementations in terms of cycles per feature. In the ideal scenario, there should be no dependence on memory latency given a sufficient amount of memory-level parallelism. We have the same simulation setup as the SpMV experiment, as well as a latency and no-latency variant of the statically-scheduled kernel. There are also several variations of the Chext implementation, which differ in the number (N) of load-add-store units. The Chext implementation uses the least-significant bits of the feature to distribute them among load-add-store units. The feature vector is sampled from a uniform integer distribution with range $[0, 2^{10} - 1]$

The results of this experiment are depicted in Figure 13. We first observe that even the most basic Chext implementation ($N = 1$) outperforms the best statically-scheduled Vitis HLS implementation. Unlike the case of SpMV, configuring the memory latency does not help the statically-scheduled design at all. The performance of the Chext implementation can be adjusted by changing the number of load-store-units. The performance improves with increasing N , as the accelerator can take advantage of more memory-level parallelism. The designer can choose N in such a way that the

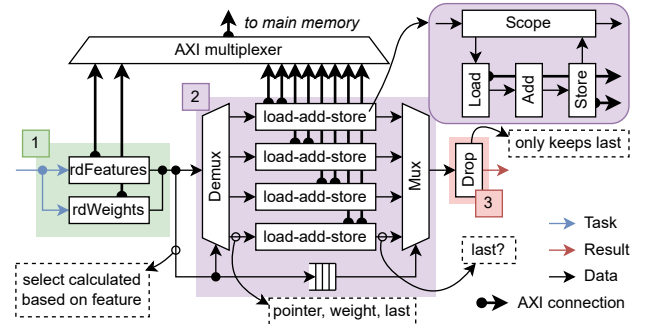


Figure 12: Architecture of the Chext-based Histogram accelerator. It leverages memory-level parallelism by employing multiple load-add-store units. An individual load-add-store unit employs a scope to avoid read-after-write hazards.

Chext implementation meets the performance requirements within resource constraints, given the synthesis results in Table 2.

Synthesis results. We present the synthesis results for the histogram accelerator in Table 2. Both Chext and Vitis HLS designs have similar resource needs. We note that the Chext variant with $N = 16$ performs $6.6\times$ to $4.8\times$ better than the best Vitis implementation, while only needing $2.1\times$ resources.

6 Related Work

Vitis HLS. Vitis HLS excels at generating high-performance statically-scheduled designs, mostly automatically [21]. Chext neither targets statically-scheduled designs nor does it provide the same level of automation. Unlike Chext, Vitis HLS can also generate floating-point cores depending on the targeted device and clock frequency. It is possible to combine Chext and Vitis HLS over elastic interfaces, in order to get the best out of both tools, as shown in Section 5.

Developing parametrized and complex dataflow architectures, like trees, in Vitis HLS is challenging using pure C/C++. In fact, numerous projects use code generation as part of their HLS flows [5, 9, 19], which is not an easy approach. Chext, in contrast, can easily generate complex architectures without using any other scripting language. It can also integrate pure RTL seamlessly.

Vitis HLS tasks are written in a sequential context, which might have unexpected consequences. For example, consider writing to

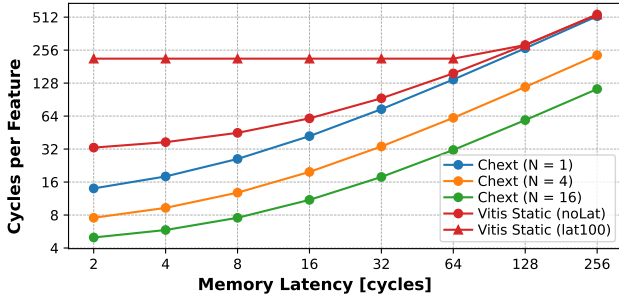


Figure 13: Performance of Chext and statically-scheduled Vitis HLS designs. Chext overperforms all Vitis HLS variants and its performance can be improved by adding more load-add-store units (i.e., increasing N).

Table 2: Resource utilizations of Chext and Vitis HLS designs. In parentheses, the same quantity is reported relative to the baseline Vitis design.

	LUT	FF	BRAM	DSP
Spmv, Vitis Static	3,818 (100%)	4,216 (100%)	1 (100%)	2 (100%)
Spmv, Chext, v.0	3,116 (82%)	1,179 (28%)	0 (0%)	2 (100%)
Spmv, Chext, v.1	4,748 (124%)	1,480 (35%)	0 (0%)	2 (100%)
Spmv, Chext, v.2	10,248 (268%)	3,374 (80%)	0 (0%)	2 (100%)
Spmv, Chext, v.Vitis	4,753 (124%)	3,080 (73%)	0 (0%)	2 (100%)
Hist, Vitis Static	1,838 (100%)	2,808 (100%)	1 (100%)	0
Hist, Chext (1)	983 (53%)	441 (16%)	0 (0%)	0
Hist, Chext (4)	1,740 (95%)	678 (24%)	0 (0%)	0
Hist, Chext (16)	3,889 (212%)	1,438 (51%)	0 (0%)	0

two independent queues. Due to the sequential nature of the C/C++ kernel, these two write operations may become coupled and result in a deadlock later in the circuit when they are read. In fact, even if the design respects the order when reading, compiler optimizations may reorder the code [20]. Fences help in such situations, but not when the order of operations is not known. Chext does not suffer from these types of deadlocks thanks to its fully-parallel nature.

Dynamic. Dynamic automatically generates elastic dataflow circuits from C/C++ programs [10]. Compared to Chext, it uses a more constrained elastic circuit approach: control and data tokens are separated, the circuit is composed of stateless components, and the program state is carried only by tokens. Chext, in contrast, has a more elaborate approach to compensate for the lack of automation. It also provides a rich set of tools for interacting with memory, including streaming components, which are absent in Dynamic.

Bluespec SystemVerilog (BSV). BSV uses a rule-based design paradigm with a scheduler to create digital circuits. It has a higher abstraction level than Chext and Chisel. In principle, it is possible to construct a Chext-like library in BSV [18]; however, it probably would not have the same ergonomics as Scala/Chisel. Regardless, BSV inspired Chext in many ways. Chext features a *transaction* component that emulates the rule-based design paradigm of BSV [7]. Of course, it is not as natural as BSV, due to the lack of a scheduler. Our queue implementation mimics BSV’s special FIFOs [15].

Low-level RTL. Writing low-level RTL is always an option, and, in fact, the guaranteed way to realize the desired design. However, compared to Chext, HLS, or BlueSpec, this approach is not productive. The designer needs to drive AXI interfaces, instantiate floating-point units, and schedule all the operations correctly. Handling all these tasks in low-level RTL is impractical. For reference, Chext designs in this paper yield 3k-20k lines of Verilog code.

Other related work. Some of the elastic components that we present in Chext were proposed by early work (from 60s and then 70s) on dataflow machines, such as loops [12, 14], albeit with different programming interfaces. Caph is another dataflow-based DSL that first coined the term data-is-control [16]. Its philosophy is similar to Chext; however, it works at a higher abstraction level without involving RTL. Maxeler MaxCompiler also targets dataflow circuits. While the information available online about this tool is limited, it seems to work at a higher abstraction level than Chext. We cannot tell what kind of circuits MaxCompiler generates circuits because we do not have access to it [13].

7 Conclusion

In this paper, we introduced Chext, a Scala-based DSL for safe and agile dataflow circuit design. Its novel approach enables creating dataflow circuits from a textual representation that directly mirrors the schematic. This RTL-like approach results in high-performance designs that exploit task-level and memory-level parallelism. We presented Chext’s design principles and built-in components, followed by case studies on two accelerators: sparse matrix-vector multiplication and histogram. In both cases, Chext-based implementations achieved better performance than the equivalent Vitis HLS designs, with similar resource utilization.

References

- [1] 2019. IEEE Standard for Floating-Point Arithmetic. *IEEE Std 754-2019 (Revision of IEEE 754-2008)* (2019), 1–84. doi:10.1109/IEEESTD.2019.8766229
- [2] 2020. *AMBA® AXI and ACE Protocol Specification (ARM IHI 0022H)*.
- [3] 2023. IEEE Standard for Standard SystemC® Language Reference Manual. *IEEE Std 1666-2023 (Revision of IEEE Std 1666-2011)* (2023), 1–618. doi:10.1109/IEEESTD.2023.10246125
- [4] Jonathan Bachrach, Huy Vo, Brian Richards, Yunsup Lee, Andrew Waterman, Rimas Avizienis, John Wawrzynek, and Krste Asanović. 2012. Chisel: Constructing hardware in a Scala embedded language. In *DAC Design Automation Conference 2012*. 1212–1221. doi:10.1145/2228360.2228584
- [5] Yuze Chi, Jason Cong, Peng Wei, and Peipei Zhou. 2018. SODA: stencil with optimized dataflow architecture. In *Proceedings of the International Conference on Computer-Aided Design* (San Diego, California) (*ICCAD '18*). Association for Computing Machinery, New York, NY, USA, Article 116, 8 pages. doi:10.1145/3240765.3240850
- [6] J. Cortadella, M. Kishinevsky, and B. Grundmann. 2006. Synthesis of synchronous elastic architectures. In *2006 43rd ACM/IEEE Design Automation Conference*. 657–662. doi:10.1145/1146909.1147077
- [7] David J Greaves. 2015. Layering RTL, SAFL, Handel-C and Bluespec constructs on Chisel HCL. In *2015 ACM/IEEE International Conference on Formal Methods and Models for CoDesign (MEMOCODE)*. 108–117. doi:10.1109/MEMCOD.2015.7340477
- [8] John R. Hauser and Andrew Waterman. [n. d.]. Berkeley Hardware Floating-Point Units. <https://github.com/ucb-bar/berkeley-hardfloat/tree/master>. Accessed: 2025-01-09.
- [9] Lana Josipovic, Nithin George, and Paolo Ienne. 2016. Enriching C-based High-Level Synthesis with parallel pattern templates. In *2016 International Conference on Field-Programmable Technology (FPT)*. 177–180. doi:10.1109/FPT.2016.7929527
- [10] Lana Josipović, Radhika Ghosal, and Paolo Ienne. 2018. Dynamically Scheduled High-level Synthesis. In *Proceedings of the 2018 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays* (Monterey, CALIFORNIA, USA) (*FPGA '18*). Association for Computing Machinery, New York, NY, USA, 127–136. doi:10.1145/3174243.3174264
- [11] Eliyah Kilada, Shomit Das, and Kenneth Stevens. 2010. Synchronous elasticization: Considerations for correct implementation and MiniMIPS case study. In *2010 18th IEEE/IFIP International Conference on VLSI and System-on-Chip*. 7–12. doi:10.1109/VLSISOC.2010.5642631
- [12] Paul R. Kosinski. 1973. A data flow language for operating systems programming. *SIGPLAN Not.* 8, 9 (Jan. 1973), 89–94. doi:10.1145/390014.808289
- [13] Oliver Pell and Vitali Averbukh. 2012. Maximum Performance Computing with Dataflow Engines. *Computing in Science & Engineering* 14, 4 (2012), 98–103. doi:10.1109/MCSE.2012.78
- [14] Jorge E Rodrigues. 1969. A graph model for parallel computations. (1969).
- [15] D.L. Rosenband. 2004. The ephemeral history register: flexible scheduling for rule-based designs. In *Proceedings. Second ACM and IEEE International Conference on Formal Methods and Models for Co-Design, 2004. MEMOCODE '04*. 189–198. doi:10.1109/MEMCOD.2004.1459853
- [16] Jocelyn Sérot and François Berry. 2019. The CAPH Language, Ten Years After. In *Embedded Computer Systems: Architectures, Modeling and Simulation (19th International Conference, SAMOS 2019)*. 336–347. doi:10.1007/978-3-030-27562-4_24
- [17] Wilson Snyder. [n. d.]. Verilator: Open Source Verilog HDL Simulator and lint System. <https://www.veripool.org/verilator>. Accessed: 2025-09-29.
- [18] Muralidaran Vijayaraghavan and Arvind. 2009. Bounded Dataflow Networks and Latency-Insensitive circuits. In *2009 7th IEEE/ACM International Conference on Formal Methods and Models for Co-Design*. 171–180. doi:10.1109/MEMCOD.2009.5185393
- [19] Jie Wang, Licheng Guo, and Jason Cong. 2021. AutoSA: A Polyhedral Compiler for High-Performance Systolic Arrays on FPGA. In *The 2021 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays* (Virtual Event, USA) (*FPGA '21*). Association for Computing Machinery, New York, NY, USA, 93–104. doi:10.1145/3431920.3439292
- [20] Xilinx. 2025. HLS Fence Library (UG1399). <https://docs.amd.com/r/en-US/ug1399-vitis-hls/HLS-Fence-Library>. Accessed: Sep 8, 2025.
- [21] Xilinx. 2025. Vitis High-Level Synthesis User Guide (UG1399). <https://docs.amd.com/r/en-US/ug1399-vitis-hls>. Accessed: Sep 8, 2025.